

CS323 Documentation

1. Problem Statement

The second assignment is to write a syntax analyzer. You may use any top-down parser such as RDP, a predictive recursive descent parser or a table-driven predictive parser.

Hence, your assignment consists of the following tasks:

1. Rewrite the grammar Rat26S to remove any left recursion (Also, use left factorization if necessary)
2. Use the lexer() generated in the assignment 1 to get the tokens
3. The parser should print to an **output file the tokens, lexemes and the production rules used**;
4. **Error handling**: if a syntax error occurs, your parser should generate a meaningful error message, such as token, lexeme, line number, and error type etc.

2. How to use your program

This program was run and tested using the g++.exe compiler with version 13.2.0. In the same directory as the location of main.cpp, please perform the following:

Step 1: How to compile (Windows): g++ -o syntax_analyzer main.cpp

How to compile (Linux): make
(or using your desired compiler)

Step 2: How to execute:

There must be 2 command-line arguments given when performing the command for executing the program. These two arguments must be the names of text files. The first argument is an already existing input file which contains Rat26S source code. This is the file that will be read from. The second argument is the output file which will contain the generated tokens, lexemes, and productions used for syntax analysis. This file will be created once the program is finished.

The format of the command is:

For Windows (use PowerShell):

`./syntax_analyzer.exe "<input_file_name>.txt" "<output_file_name>.txt"`
ex) `./syntax_analyzer.exe "input_test_file_1.txt" "output_test_file_1.txt"`

For Linux:

`./syntax_analyzer "<input_file_name>.txt" "<output_file_name>.txt"`
ex) `./syntax_analyzer "input_test_file_1.txt" "output_test_file_1.txt"`

The created output .txt file should now appear in your current directory with the name you gave it, and it should contain tokens, lexemes, and productions in the Rat26S grammar.

3. Design of your program

Selected Programming Language: C++

Selected Top Down Parser Implementation: Predictive Recursive Descent Parser (PRDP)

Source files that were included when compiling this program:

- main.cpp
- fsm.hpp (Assignment 1)
- lexical_analyzer.hpp (Assignment 1)
- prdp.hpp (Assignment 2)

Important Note: We have rewritten the provided Rat26S grammar in order to remove any Left Recursive Productions and prevent Backtracking. Below are the rules that were rewritten:

Rules in the Rat26S Grammar that were altered:

R3. <Function Definitions> ::= <Function> | <Function> <Function Definitions>

<Function Definitions> ::= <Function> <Function Definitions Prime>
<Function Definitions Prime> ::= <Function Definitions> | <Empty>

R6. <Parameter List> ::= <Parameter> | <Parameter> , <Parameter List>

<Parameter List> ::= <Parameter> <Parameter List Prime>
<Parameter List Prime> ::= , <Parameter List> | <Empty>

R11. <Declaration List> ::= <Declaration> ; | <Declaration> ; <Declaration List>

<Declaration List> ::= <Declaration> ; <Declaration List Prime>
<Declaration List Prime> ::= <Declaration List> | <Empty>

R13. <IDs> ::= <Identifier> | <Identifier> , <IDs>

<IDs> ::= <Identifier> <IDs Prime>
<IDs Prime> ::= , <IDs> | <Empty>

R14. <Statement List> ::= <Statement> | <Statement> <Statement List>

<Statement List> ::= <Statement> <Statement List Prime>
<Statement List Prime> ::= <Statement List> | <Empty>

R18. <If> ::= if (<Condition>) <Statement> fi | if (<Condition>) <Statement> otherwise <Statement> fi

<If> ::= if (<Condition>) <Statement> <If Prime>
<If Prime> ::= fi | otherwise <Statement> fi

R19. <Return> ::= return ; | return <Expression> ;

<Return> ::= return <Return Prime>
<Return Prime> ::= ; | <Expression> ;

R25. <Expression> ::= <Expression> + <Term> | <Expression> - <Term> | <Term>

<Expression> ::= <Term> <Expression Prime>
<Expression Prime> ::= + <Term> <Expression Prime> | - <Term> <Expression Prime> | <Empty>

R26. <Term> ::= <Term> * <Factor> | <Term> / <Factor> | <Factor>

<Term> ::= <Factor> <Term Prime>
<Term Prime> ::= * <Factor> <Term Prime> | / <Factor> <Term Prime> | <Empty>

R28. <Primary> ::= <Identifier> | <Integer> | <Identifier> (<IDs>) | (<Expression>) |
<Real> | true | false

<Primary> ::= <Identifier> <Primary Prime> | <Integer> | (<Expression>) | <Real> | true | false
<Primary Prime> ::= (<IDs>) | <Empty>

main.cpp

The main function of the program takes care of initiating vital tasks. First, it initiates lexical analysis by calling the lexical analyzer which creates a list of Record objects that contain tokens, lexemes, and the line number that these items were located at in the Rat26S input source file. This list of tokens and lexemes is then passed to the parser via an object called Rat26SParser. We then initiate syntax analysis and begin parsing via the `write_productions_to_file( )` member function that belongs to the Rat26SParser class.

prdp.hpp

This header file contains our implementation for a Predictive Recursive Descent Parser (PRDP). Rat26SParser is the name given to our class which encapsulates all the data and behaviors that are required for syntax analysis. There are around more than 30 member functions defined in our class where a majority consist of simulating a production listed in the grammar for the Rat26S programming language. Each nonterminal that exists in the Rat26S grammar's set of productions has a corresponding function.

Below is the format we used to create all the Rat26SParser member functions that simulate the productions in the grammar:

Production form: `<nonterminal_symbol> ->  $\beta$` where β is a sentential containing a mix of terminals and non-terminal symbols.

Function form:

```
void <nonterminal_symbol>( )
{
    // Consume tokens that  $\beta$  consists of using the member function lexer( )
    // Make function calls corresponding with nonterminal symbols that
    // appear in  $\beta$ 
}
```

The `lexer( )` member function simply moves the index variable for the list of tokens in order to obtain the next token. The only time that this function is called is when a terminal symbol is seen in the current production that we are looking at. One important note is that `lexer( )` does not output the token but instead “consumes” the current token that is being analyzed. Additionally, the token and lexeme are written to the output file only when they are detected to be in the FIRST set of a nonterminal. If we have just moved to the next token and we immediately find a

terminal symbol in our current production, then we write to the output file and call `lexer( )` to consume the newly found token.

As mentioned previously, the member function that starts the chain of calls is `write_productions_to_file( )`. This function starts the chain by calling the member function that corresponds with the starting nonterminal symbol in the Rat26S grammar. This function is called `Rat26S( )` which simulates the very first rule in the grammar.

Error Handling

If a violation of a production occurs, then we output a syntax error which includes relevant information regarding the violation. This information includes the token, lexeme, line number, what was expected, and the production that was violated. Below is an example of what would be written to the output file if a syntax error occurred:

Syntax Error in `input_test_file_1.txt` on line 1 with

Token : keyword

Lexeme : if

Expected : @

Violation of: `<Rat26S> ::= @ <Opt Function Definitions> @ <Opt Declaration List> @ <Statement List> @`

4. Any Limitation

'None'

5. Any shortcomings

'None'